

Teste Técnico – Sistema de Registro de Ponto

Desenvolver uma aplicação completa de registro de jornada de trabalho composta por:

- API Backend em Spring Boot
- Aplicação Frontend em Flutter
- Banco de dados relacional utilizando MySQL ou PostgreSQL
- Automação RPA em Python integrada ao sistema

1. Arquitetura da Solução

Backend

- Java (versão mais recente)
- Spring Boot
- Swagger/OpenAPI
- Banco de dados MySQL ou PostgreSQL

Responsabilidades Backend

- Controle de jornadas
- Regras de negócio
- Persistência de dados
- Relatórios consolidados
- API REST
- Recebimento de dados do RPA

Frontend

- Flutter
- Dart
- Provider ou Riverpod para gerenciamento de estado
- Consumo de API REST

Responsabilidades Frontend

- Interface do colaborador
- Interface da gestão
- Histórico de jornadas
- Relatórios
- Visualização dos dados importados pelo RPA

Banco de Dados

- colaboradores
- jornadas
- atividades_planejadas
- atividades_realizadas
- atividades_nao_planejadas
- registros_rpa

2. Regras de Negócio

2.1 Início da Jornada (Check-in)

- Data/hora de início
- Lista de atividades planejadas
- Não permitir iniciar nova jornada com outra em andamento
- Status: EM_ANDAMENTO e FINALIZADA
- Atividades vinculadas à jornada

2.2 Encerramento da Jornada (Check-out)

- Marcar atividades planejadas como concluídas
- Adicionar atividades não planejadas
- Registrar horário de encerramento
- Registrar duração da jornada
- Registrar resumo das atividades
- Apenas jornadas EM_ANDAMENTO podem ser encerradas
- Após encerrada nenhuma informação poderá ser alterada
- Garantir integridade transacional

2.3 Relatórios Consolidados

- Total de horas trabalhadas
- Quantidade de atividades planejadas
- Quantidade de atividades concluídas
- Quantidade de atividades não planejadas
- Percentual de aderência

3. Frontend – Funcionalidades Obrigatórias

3.1 Área do Colaborador

- Iniciar Jornada

- Informar atividades planejadas
- Registrar horário de entrada automaticamente
- Encerrar Jornada
- Marcar atividades concluídas
- Informar atividades não planejadas
- Registrar horário de saída automaticamente
- Histórico com data, entrada, saída, total de horas e aderência

3.2 Área de Gestão

- Gestão de colaboradores
- Listagem de colaboradores
- Consulta de jornadas
- Relatórios consolidados
- Indicadores gerais
- Filtros
- Resumo estatístico

4. Automação RPA em Python

4.1 Objetivo

- Abrir navegador real
- Acessar Portal Ponto Ágil
- Navegação dinâmica
- Web scraping real
- Processamento dos dados
- Envio para API
- Persistência no banco
- Exibição no frontend

4.2 Requisitos Obrigatórios do RPA

- Execução visual
- Acompanhamento em tempo real
- Demonstração clara da navegação
- Coleta de informações
- Tratamento dos dados
- Envio para API
- Execução independente via comando: `py main.py`

5. Estrutura de Repositórios

Repositórios

- registro-ponto-backend
- registro-ponto-frontend
- registro-ponto-rpa

6. Controle de Acesso aos Repositórios

Compartilhamento

- Compartilhar com usuário: playercontabilidade

README

- Requisitos
- Configuração do ambiente
- Variáveis de ambiente
- Execução do backend
- Execução do frontend
- Execução do RPA
- Credenciais de teste
- Fluxo da aplicação

7. Critérios de Avaliação

Avaliação

- Organização do código
- Arquitetura
- Qualidade das regras de negócio
- Clareza da API
- Qualidade do frontend
- Estrutura do banco
- Funcionamento do RPA
- Integração entre sistemas
- Documentação
- Boas práticas de desenvolvimento e versionamento